

Lab 4

TensorFlow + CNN

Klasyfikacja obrazów z użyciem konwolucyjnych sieci neuronowych

Wybrane Zagadnienia Sztucznej Inteligencji

Informatyka Stosowana · II stopień · 2025/2026

Co dzisiaj robimy

1

TensorFlow

Framework który robi backprop za nas

3

MLP w TensorFlow

To samo co Lab 3, ale 3 linijki kodu

5

Konwolucja

Stride, padding, pooling, flatten

7

ZADANIE — CNN dla MNIST

Architektura LeNet-style, ~99% accuracy

2

Tensory

Skalar, wektor, macierz, tensor ND

4

CNN — idea

Sieci które rozumieją obrazy

6

Funkcje aktywacji

ReLU, LeakyReLU, GeLU, softmax

8

Zadania dodatkowe

Optymalizatory, regularyzacja, augmentacja

1998

Yann LeCun · Bell Labs

Architektura, którą za chwilę zbudujecie, ma 28 lat.



LeNet-5

Dwie warstwy konwolucyjne, dwie poolingowe, dwie gęste — dokładnie ten szkielet, który wy zbudujecie dzisiaj. W szczycie czytał ~20% wszystkich czeków bankowych USA.

„To, co zbudujesz dziś, to bezpośredni potomek modelu z 1998. Trening: 6 dni → 2 minuty. Skok: ~1000× w 14 lat.”

TensorFlow w jednym zdaniu

Biblioteka do **obliczeń tensorowych z automatycznym różniczkowaniem** i **akceleracją sprzętową**.

Tensorowe

Operacje na N-wymiarowych tablicach liczb (tensorach). Optymalizowane do równoległości.

Auto-różniczkowanie

TensorFlow buduje graf obliczeń i sam liczy pochodne. To, co robiliśmy ręcznie na Lab 3 — on robi za nas.

Akceleracja sprzętowa

Kompiluje do CUDA (GPU), XLA (TPU), AVX (CPU). Te same 3 linijki kodu działają wszędzie.

TensorFlow vs PyTorch w 2026

PyTorch dominuje w research (Meta, OpenAI, Anthropic). **TensorFlow** trzyma się w produkcji enterprise — telefony, edge, TFLite. Składnia 90% wspólna — kto umie jedno, w weekend opanuje drugie.

Tensor = N-wymiarowa tablica liczb

Wymiar	Nazwa	Przykład	Zastosowanie
0D	Skalar	5.0	learning rate, loss value
1D	Wektor	[1, 2, 3]	cechy jednej próbki, wagi neuronu
2D	Macierz	[[1,2],[3,4]]	batch wektorów, obraz grayscale
3D	Tensor 3D	(H, W, 3)	obraz RGB
4D	Tensor 4D	(batch, H, W, C)	<i>batch obrazów — CNN!</i>

Format 4D dla CNN

CNN potrzebuje 4D: (**batch, height, width, channels**). Np. (32, 28, 28, 1) dla MNIST. MLP wystarczyło 2D — to pierwsza praktyczna różnica.

Lab 3 → dzisiaj: ta sama matematyka, 5× mniej kodu

Lab 3 · numpy od zera

```
# Forward pass
z1 = X @ W1 + b1
a1 = relu(z1)
...

# Backward pass
dz2 = a2 - y
dW2 = a1.T @ dz2
...

# Update
W1 -= lr * dW1
...
```

~50 linijek numpy

Dziś · TensorFlow

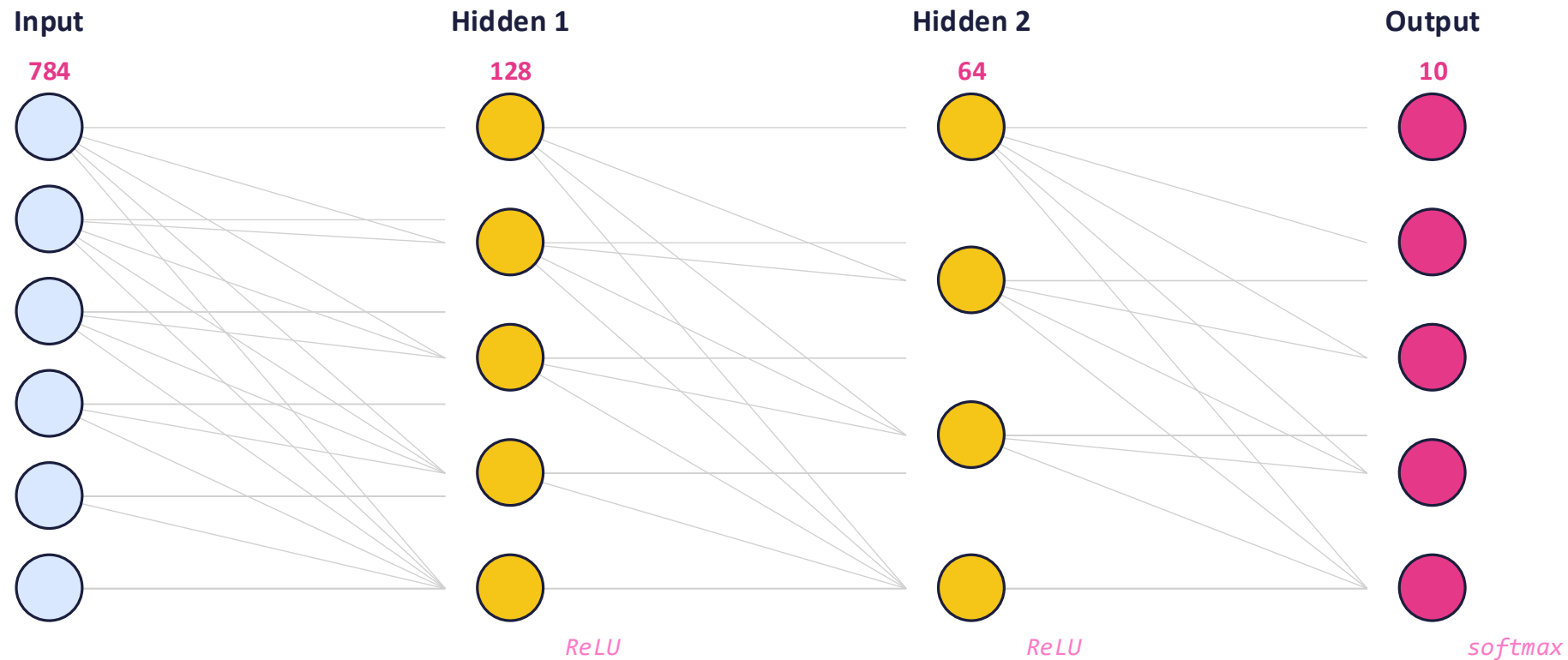
```
model = tf.keras.Sequential([
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', ...)
model.fit(X, y, epochs=5)
```

~3 linijki architektury

To NIE jest oszustwo — to ta sama matematyka. Sens Lab 3: rozumiesz CO TensorFlow robi pod maską. Bo gdy się popsuje — musisz to zdebugować.

Architektura MLP dla MNIST



$28 \times 28 = 784$ pikseli (spłaszczony obraz) $\rightarrow 128 \rightarrow 64 \rightarrow 10$ klas (cyfry 0-9)

Pierwsza warstwa Dense ma $784 \times 128 + 128 = 100\,480$ parametrów

Trenujemy MLP — co widzimy w notebooku

5 epok, batch_size=32, validation_split=0.2

```
Epoch 1/5
1500/1500 [=====] - 3s 2ms/step - accuracy: 0.8705 - loss: 0.4513 - val_accuracy: 0.9565
Epoch 2/5
1500/1500 [=====] - 2s 2ms/step - accuracy: 0.9651 - loss: 0.1171 - val_accuracy: 0.9657
Epoch 3/5
1500/1500 [=====] - 2s 2ms/step - accuracy: 0.9758 - loss: 0.0780 - val_accuracy: 0.9653
Epoch 4/5
1500/1500 [=====] - 2s 1ms/step - accuracy: 0.9829 - loss: 0.0561 - val_accuracy: 0.9723
Epoch 5/5
1500/1500 [=====] - 2s 2ms/step - accuracy: 0.9876 - loss: 0.0401 - val_accuracy: 0.9750
```

Test accuracy

~97.5%

Z 10 000 testów myli się ~250 razy.

Dla MNIST w 2026 to słabo. Za chwilę CNN da ~99%.

Dlaczego MLP nie wystarczy dla obrazów

1

Tracimy strukturę 2D

MLP spłaszcza obraz w wektor. Dla niego piksel (0,0) i (27,27) to po prostu pozycje 0 i 783. Sąsiedztwo nie ma znaczenia.

2

Nie skaluje się

MNIST $28 \times 28 \rightarrow$ ok, 100k parametrów. CIFAR $32 \times 32 \times 3 \rightarrow$ 1.5M. ImageNet $224 \times 224 \times 3 \rightarrow$ 77M tylko w pierwszej warstwie. Nie do trenowania.

3

Brak odporności na przesunięcia

Wytrenuj na wycentrowanych cyfrach, daj cyfrę przesuniętą o 5 pikseli — MLP się gubi. CNN — nie.

CNN rozwiązuje wszystkie trzy →

CNN opiera się na trzech ideach



Sparse interactions

Rzadkie połączenia

Neuron CNN patrzy tylko na lokalny region wejścia, nie na cały obraz.

Mniej połączeń, naturalna lokalność.



Parameter sharing

Dzielenie wag

Ten sam filtr przesuwają się po całym obrazie — te same wagi w wielu pozycjach.

1000× mniej parametrów + ekwiwariantność.



Equivariant representation

Reprezentacja ekwiwariantna

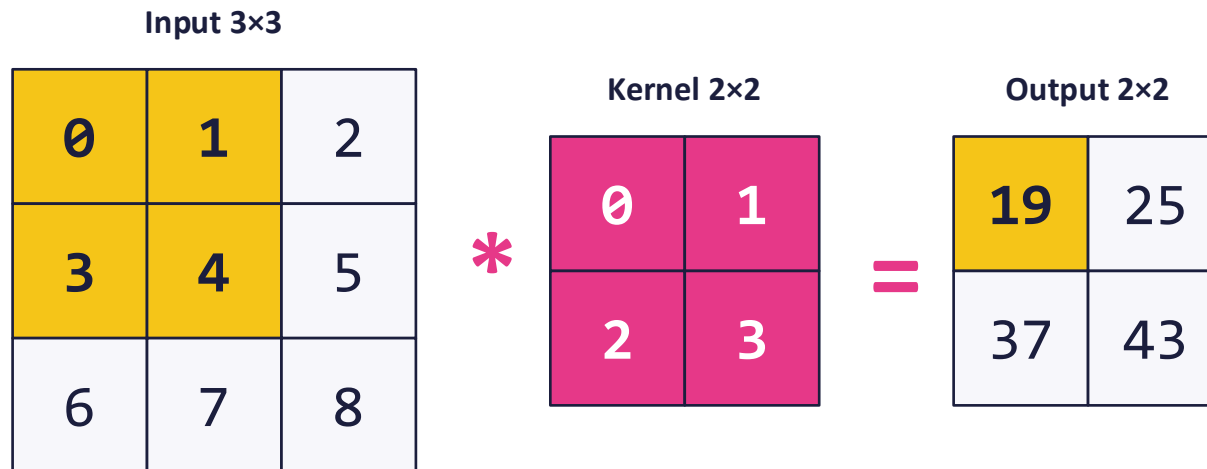
Wzór przesunięty w obrazie = jego reprezentacja przesunięta w feature mapie.

Sieć rozpoznaje cyfrę niezależnie od pozycji.

S — P — E. Albo prościej: lokalne · dzielone · przesuwne.

Konwolucja — krok po kroku

Bierzesz kernel, przykładasz na region, mnożysz element po elemencie, sumujesz.



Jak powstaje 19?

$$19 = 0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 0 + 1 + 6 + 12 = 19$$

Pomarańczowe pola = aktualna pozycja kernela. Przesuwasz o 1 piksel w prawo, powtarzasz dla każdej pozycji.

Sobel — ręcznie zaprojektowany filtr

Filtry wykrywające krawędzie istnieją od 1968. Patrzcie, co potrafią ZROBIĆ — i co CNN robi automatycznie.

Sobel X (pionowe krawędzie)

-1	0	1
-2	0	2
-1	0	1

Sobel Y (poziome krawędzie)

-1	-2	-1
0	0	0
1	2	1

1968

Ludzie projektowali takie filtry RĘCZNIE.

Ktoś przez 5 lat pracował nad detektorem rogu. Doktoraty z computer vision powstawały o tym, jak zaprojektować jeden konkretny filtr.

2012

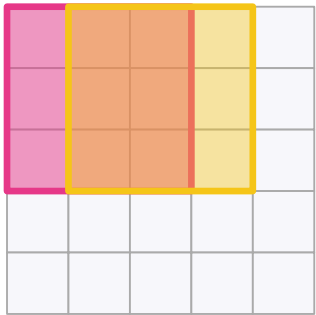
AlexNet SAM się ich nauczył w 6 dni.

Wystarczyło dać sieci dużo obrazów i powiedzieć „odróżniaj psy od kotów”. Pierwsze warstwy CNN automatycznie odkryły filtry podobne do Sobela. To „deep” w deep learning — model sam uczy się hierarchii reprezentacji.

Stride — o ile przesuwa się kernel

Stride = 1

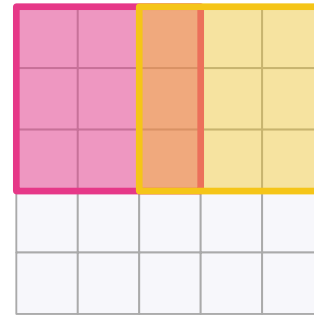
analizujemy każdą pozycję



Output: 3×3

Stride = 2

przeskakujemy co drugi piksel



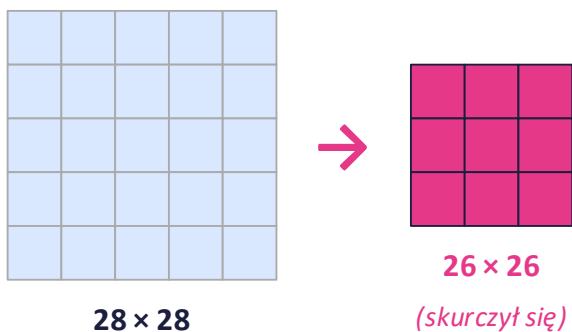
Output: 2×2

Po co stride > 1?

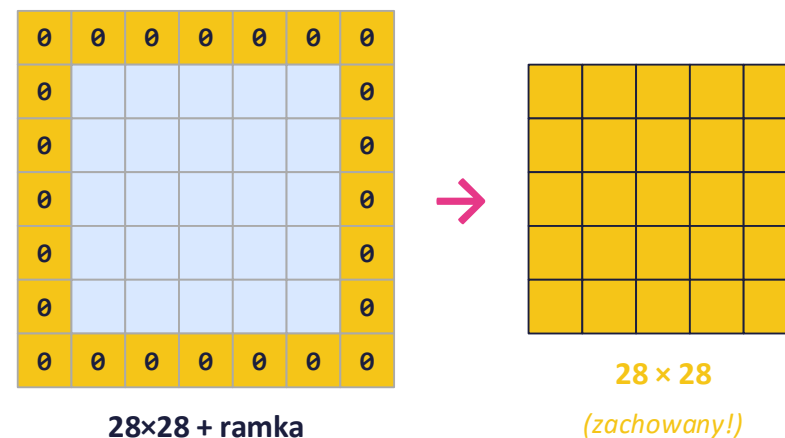
Stride 2 daje output 2× mniejszy → mniej obliczeń w głębszych warstwach. To alternatywa dla pooling layer (nowsza praktyka). W naszym zadaniu używamy domyślnego stride=1.

Padding — zera na krawędziach

Bez paddingu (valid)



Z paddingiem=1 (same)



padding='valid' (default w naszym zadaniu) vs padding='same'

'valid' — konwolucja tylko gdzie cały kernel mieści się w obrazie (output mniejszy). 'same' — dodaj zera, żeby output miał ten sam rozmiar co input. Zachowany rozmiar = możesz stackować dużo warstw bez kurczenia.

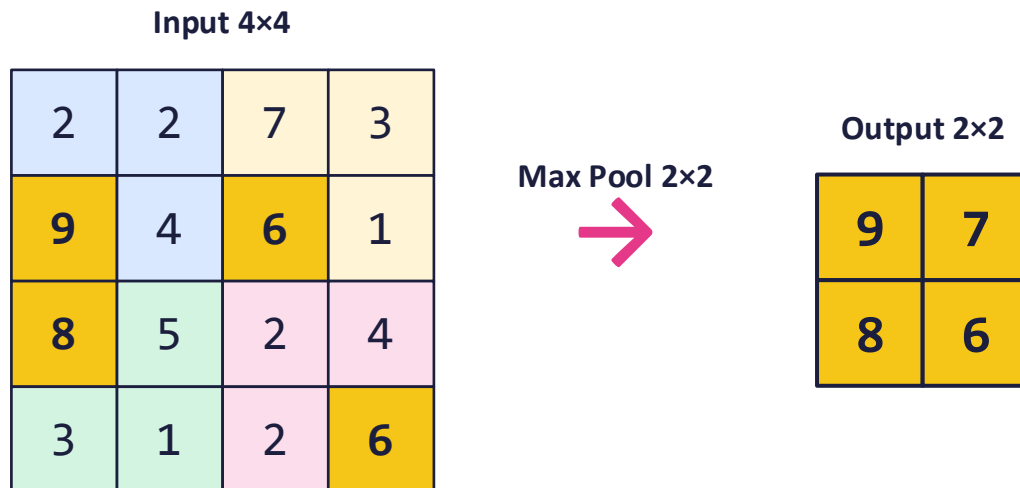
Wzór na rozmiar wyjścia

$$\text{output} = (\text{input} + 2 \cdot \text{padding} - \text{kernel}) / \text{stride} + 1$$

input	kernel	padding	stride	output	po co?
28	3	0	1	26	nasz Conv2D #1 — domyślne, kurczy się o 2
28	3	1	1	28	padding='same' — zachowuje rozmiar
28	3	0	2	13	stride 2 — redukcja 2×
26	2	0	2	13	MaxPool 2×2 — to też ten wzór!

⚠ PDF instrukcji ma ten wzór bez nawiasów — pamiętaj: **NAPIERW** odejmujesz, **POTEM** dzielisz.

Pooling — kompresja feature mapy



Max pooling

Maksimum z okna — zachowuje silne aktywacje. Najpopularniejsze.

Avg pooling

Średnia z okna — zachowuje więcej szczegółów. Rzadziej w hidden, częściej na końcu (GAP).

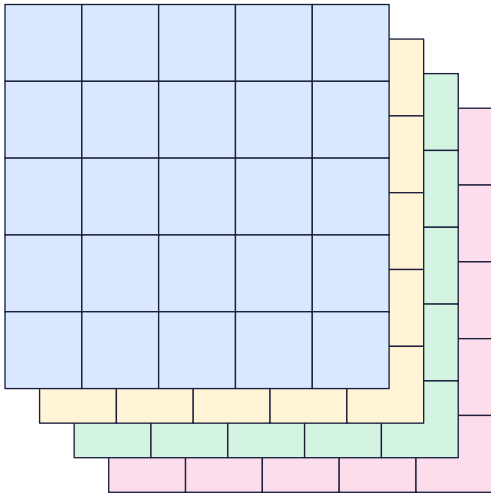
Pooling NIE MA WAG — to deterministyczna operacja matematyczna.

Trzy cele: ① redukcja wymiarów (mniej obliczeń) ② spatial invariance (odporność na małe przesunięcia) ③ większe pole recepcyjne w głębszych warstwach.

Flatten — most między CNN a Dense

Wyjście Conv+Pool

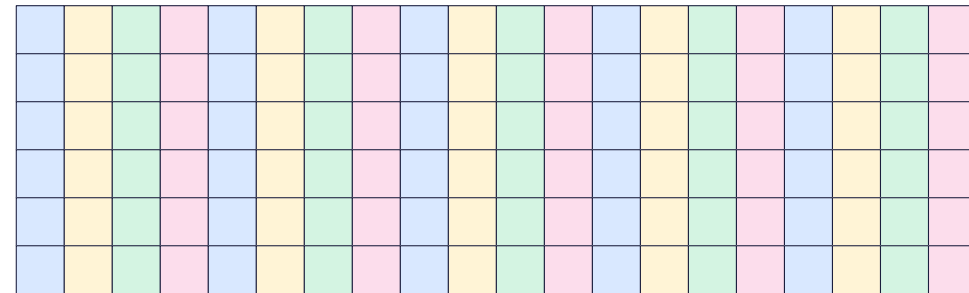
$5 \times 5 \times 36$



Flatten
→

Wektor 1D

$(900,)$

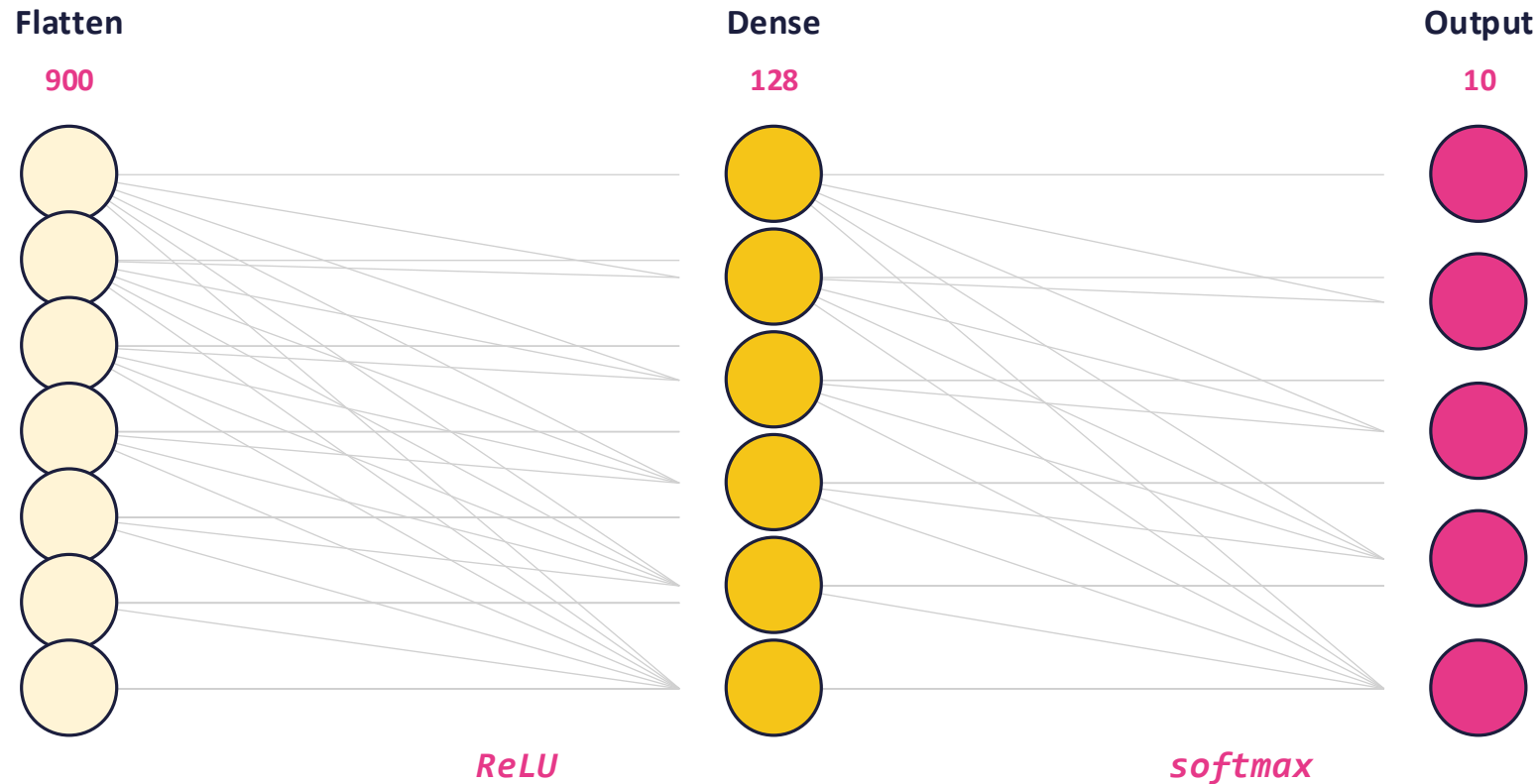


... łącznie 900 wartości

$$5 \times 5 \times 36 = 900$$

Stąd 900 jako rozmiar wejścia pierwszej warstwy Dense w naszej architekturze. Flatten nie ma wag — to czysty reshape.

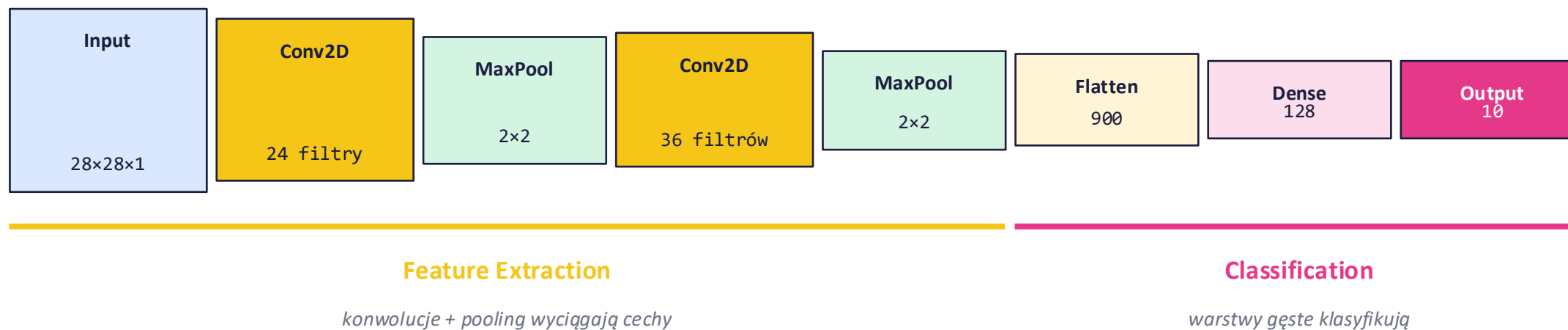
Fully connected — klasyfikator na końcu



Dense 900→128 ma **115 328** parametrów ($900 \cdot 128 + 128$).

To >92% wszystkich parametrów sieci. Konwolucje są tanie — Dense kosztuje. W nowych architekturach Dense zastępuje się Global Average Pooling.

Pełna architektura CNN — od pikseli do klasy



Wymiary feature map:

$(28, 28, 1) \rightarrow (26, 26, 24) \rightarrow (13, 13, 24) \rightarrow (11, 11, 36) \rightarrow (5, 5, 36) \rightarrow (900) \rightarrow (128) \rightarrow (10)$

Funkcje aktywacji w CNN

Nazwa	Wzór	Kiedy
ReLU	$\max(0, x)$	Domyślna w warstwach ukrytych. Tania, pochodna 0 lub 1, brak vanishing gradient.
Leaky ReLU	$x > 0 ? x : \alpha x$	Wariant gdy ReLU 'umiera' (neurony utykają w zerze). Małe nachylenie dla $x < 0$.
GeLU	$x \cdot \Phi(x)$	Gładsza wersja. Używana w BERT, GPT. Dla CNN nie istotnie lepsza.
Softmax	$e^x / \sum e^x$	TYLKO na warstwie wyjściowej. Logity → prawdopodobieństwa sumujące się do 1.

⚠ **NIGDY nie używaj softmax w warstwach ukrytych. Softmax to operacja wyjściowa, nie aktywacja w klasycznym sensie.**

Zadanie główne

CNN dla klasyfikacji cyfr MNIST

Architektura

- Input: obraz $28 \times 28 \times 1$ (MNIST, grayscale)
- Conv2D: 24 filtry 3×3 , ReLU $\rightarrow$ MaxPool 2×2
- Conv2D: 36 filtrów 3×3 , ReLU $\rightarrow$ MaxPool 2×2
- Flatten
- Dense: 128 neuronów, ReLU
- Output: Dense 10 neuronów, softmax

Trening: 10 epok · optimizer Adam · validation_split=0.2 · batch_size=32

Oczekiwany wynik: ~99% test accuracy. MLP miało ~97.5%.

Notebook: komórki 25–30. Następujący slajd: przebieg wymiarów warstwa po warstwie.

Po treningu porównacie z MLP — różnica wydaje się mała (~1.5pp), ale to spadek liczby błędów z 250 do 80.

Przebieg wymiarów warstwa po warstwie

Warstwa	Wejście	Wyjście	Parametry
Input	–	(28, 28, 1)	0
Conv2D 24, 3×3 + ReLU	(28, 28, 1)	(26, 26, 24)	240
MaxPool 2×2	(26, 26, 24)	(13, 13, 24)	0
Conv2D 36, 3×3 + ReLU	(13, 13, 24)	(11, 11, 36)	7 812
MaxPool 2×2	(11, 11, 36)	(5, 5, 36)	0
Flatten	(5, 5, 36)	(900,)	0
Dense 128 + ReLU	(900,)	(128,)	115 328
Dense 10 + Softmax	(128,)	(10,)	1 290
łącznie			~124 670 parametrów

Najtłustsza warstwa = Dense 900→128, 92% wszystkich parametrów. Conv2D są tanie.

CNN w TensorFlow — pełny kod modelu

```
model_cnn = tf.keras.models.Sequential([
    # Pierwsza warstwa konwolucyjna – 24 filtry 3x3
    tf.keras.layers.Conv2D(24, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPool2D(pool_size=(2, 2)),

    # Druga warstwa konwolucyjna – 36 filtrów 3x3
    tf.keras.layers.Conv2D(36, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPool2D(pool_size=(2, 2)),

    # Spłaszczenie – z 3D do 1D
    tf.keras.layers.Flatten(),

    # Fully-connected
    tf.keras.layers.Dense(128, activation='relu'),
    # Warstwa wyjściowa – 10 klas, softmax
    tf.keras.layers.Dense(10, activation='softmax')
])
```

Komórka 26 w notebooku · Po zbudowaniu wywołaj `model.summary()` i SPRAWDŹ wymiary z poprzedniego slajdu.

Trening 10 epok + ewaluacja

```
model_cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])  
history_cnn = model_cnn.fit(x_train_cnn, y_train_oh, epochs=10, batch_size=32, validation_split=0.2)
```

CNN Test accuracy

~99.0%

Spadek liczby błędów

3x

z 250 do ~80 błędów na 10 000

Porównanie MLP vs CNN

MLP: ~97.5% · **CNN: ~99.0%** · $\Delta = +1.5\text{pp}$

Na trudniejszych datasetach (CIFAR, ImageNet) różnica MLP vs CNN to 30+ punktów procentowych.

Zadania rozszerzające — wybierz minimum 2

A

Porównanie optymalizatorów

SGD, RMSprop, Adam. Sprawdź różne learning rate. Adam wygrywa w 90% przypadków.

B

Modyfikacja architektury

Zmień liczbę filtrów (32, 64 zamiast 24, 36). Albo dodaj więcej warstw. Sprawdź wpływ.

C

Inne funkcje aktywacji

Zamień ReLU na LeakyReLU lub ELU. Dla MNIST różnica subtelna — ale poznaj składnię.

D

Regularyzacja

Dropout (np. 0.5) lub L2 (0.001/0.01). Sprawdź wpływ na overfitting.

E

Augmentacja danych

RandomRotation, RandomTranslation, RandomZoom. Najbardziej praktyczne — efekt widoczny.

Kod do skopiowania (komórki 33-41)

A — optymalizatory

```
optimizers = {
    'SGD': tf.keras.optimizers.SGD(learning_rate=0.01),
    'RMSprop': tf.keras.optimizers.RMSprop(0.001),
    'Adam': tf.keras.optimizers.Adam(0.001),
}
```

C — LeakyReLU jako osobna warstwa

```
tf.keras.layers.Conv2D(24, (3,3), activation=None),
tf.keras.layers.LeakyReLU(alpha=0.1),
```

E — augmentacja jako pierwsza warstwa

```
data_aug = tf.keras.Sequential([
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

D — Dropout (po Dense)

```
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dropout(0.5), # 50% wylacza
```

D — L2 regularyzacja

```
tf.keras.layers.Dense(128, activation='relu',
    kernel_regularizer=
tf.keras.regularizers.l2(0.001)),
```

B — większe filtry

```
Conv2D(32, (3,3), activation='relu')
Conv2D(64, (3,3), activation='relu')
```

Typowe błędy — symptom → fix

Symptom (error / objaw)	Fix
"expected ndim=4, got ndim=3"	Zapomniany reshape. <code>x_train.reshape(-1, 28, 28, 1)</code>
Loss = NaN	Brak normalizacji (<code>/255</code>). Albo learning rate za duży.
"model has not been built yet"	Brak <code>input_shape</code> w pierwszej warstwie Conv2D.
Bardzo niska accuracy (~10%)	Brak softmax na ostatniej warstwie. Albo y nie w one-hot.
val_loss rośnie od epok ~3+	Overfitting. Dodaj Dropout albo regularyzację.
Trening trwa 10+ min/epokę	Brak GPU. Zmniejsz epochs do 3, dotrenuj w domu.

Sprawdź, co umiesz

1 Co to jest tensor i czym różni się 0D od 4D?

2 Dlaczego MLP jest gorsze niż CNN dla obrazów?

3 Co robi konwolucja — jakie 3 idee? (S, P, E)

4 Po co istnieje pooling skoro mamy już konwolucję?

5 Co robi warstwa Flatten? Dlaczego jest potrzebna?

6 Czemu ReLU jest popularniejsze niż sigmoid w CNN?

7 Co robi softmax na warstwie wyjściowej?

8 Jedna epoka CNN trwa dłużej niż MLP — dlaczego?

Jedno zdanie do zapamiętania

CNN

Conv2D wyciąga lokalne cechy, **Pooling** kompresuje, **Flatten** łączy z Dense, **Softmax** mówi „to jest cyfra 7 z 99% pewnością”.

TensorFlow robi backpropagation za nas — **nasza praca to architektura**.

Trzy idee CNN · 4D dla obrazów · architektura piramidalna — to wystarczy.

Co dalej w deep learningu na obrazach

Transfer learning

Zamiast trenować od zera, weź wytrenowaną sieć (ResNet, EfficientNet) i dostosuj pod swój problem. Standard w 2026.

Większe datasety

CIFAR-10 (60k obrazów, 10 klas, 32×32 RGB). ImageNet (14M, 1000 klas). To „prawdziwe” poligony CNN.

Głębsze architektury

ResNet (skip connections — sieci z 152+ warstwami), Inception, EfficientNet. Każda przynosi nowe pomysły.

Vision Transformers

ViT (2020) — transformer (z NLP) zaaplikowany do obrazów. Powoli wypiera CNN w nowych projektach.

Dzięki za laboratorium · powodzenia z zadaniami dodatkowymi